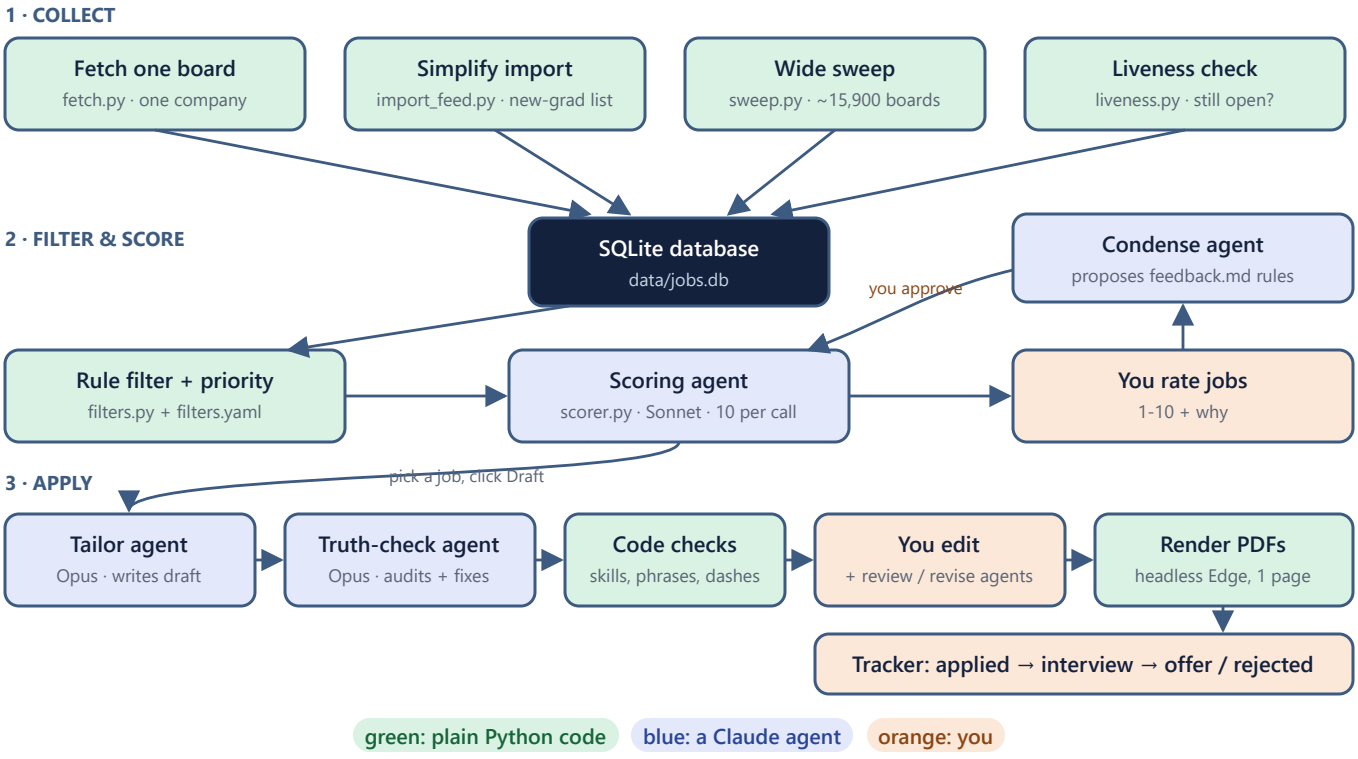
Job Sourcer

How the system works as of October 7, 2026: the pipeline, the agents, the rules and how each rule is enforced.

Job Sourcer is a personal job-search tool that runs on your PC. It pulls job postings from public ATS job boards (Greenhouse, Lever, Ashby), throws out obvious mismatches with cheap rules, and has Claude score the rest against your background. You can rate the results, and from any job it drafts a tailored one-page resume and cover letter as PDFs. A tracker follows each application from the draft to the outcome. You run all of it from a local web app, and a scheduled task does the collecting and scoring every night at 2 AM.

3,880	326	777	1,299	8
jobs in the database	scored by Claude	waiting to be scored	filtered out by rules	application drafts

The big picture



Three ideas that hold the design together

- **Cheap work before expensive work.** The rule filter is free and instant, and a Claude call uses your plan's quota. So everything that can be decided with a keyword is decided before Claude sees the job.
- **Never trust the model blindly.** Each agent returns JSON, Python checks its shape, and the things code can verify (quotes really in the posting, skills really in your profile, no em dashes) are verified in code and not just requested in the prompt.
- **You stay in charge of what's "you".** Agents can propose scoring rules but can't write `feedback.md`. Only your own words count as your writing voice, and every draft passes through you before it becomes a PDF.

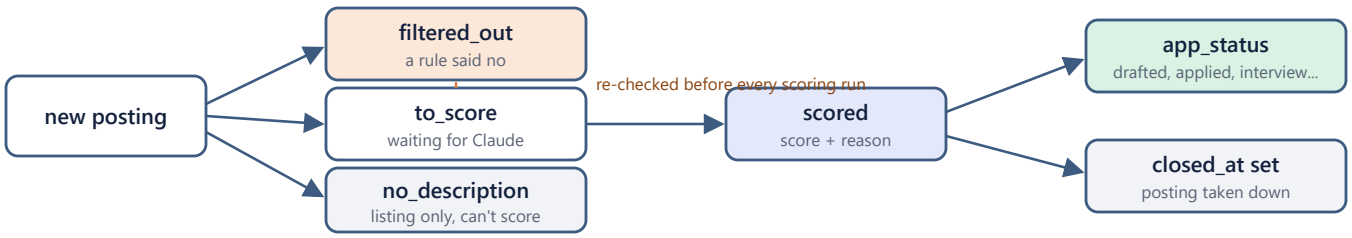
1 · Collecting jobs

Every source turns its postings into one common `Job` shape (`job.py`): source, company, job id, title, location, url, and description as plain text. Each ATS has a small adapter in `sources/` that knows its API.

Source	What it does	Politeness / limits
Fetch `fetch.py`	Pulls every posting from one company's board (Greenhouse, Lever or Ashby).	One request per board.
Simplify import `import_feed.py`	Reads SimplifyJobs' New-Grad `listings.json` for the categories in `feeds.yaml` (Software, AI/ML/Data). Postings on a supported ATS get their full description fetched. The rest are saved as listing-only (status `no_description`).	Small delay between description fetches.
Wide sweep `sweep.py`	Checks ~15,900 public boards (slugs from Bob's Job Search Kit). Keeps a job only if it's new, posted in the last 7 days, its title has a sweep keyword (new grad, junior, data analyst...), and it passes `filters.yaml`.	8 boards at a time. Boards that 404 go into `dead_boards.json` and are skipped next time.
Liveness `liveness.py`	Asks the ATS API again whether your best jobs (score or rating 7+, up to 40) are still open. Closed ones get `closed_at` set.	One request per job.

How the database stays clean: saving is an upsert (`INSERT ... ON CONFLICT DO UPDATE`) keyed on source + company + job id, so seeing a job twice updates it and never duplicates it. When a board no longer lists a job, it's marked closed. `closed_at` is kept separate from `status`, so a closed job keeps its score and history.

The life of a job (its `status`)



The filter re-runs on every unscored job before each scoring run, so a `filters.yaml` edit can move jobs between filtered_out and to_score.

2 · The rule filter (no AI)

`filters.py` reads `filters.yaml`. It is meant to be **permissive**: it removes only obvious mismatches, and anything borderline goes to Claude, which can give it a low score.

Rule	How it's enforced
Title must include a target word (engineer, developer, data, analyst, scientist, ml, ai, bi, quantitative...)	Whole-word regex `(?<!\w)word(?!\w)`, case-insensitive. "data" matches "Data Engineer" but not "Metadata".

Title must not include a seniority/wrong-field word (senior, sr, staff, principal, director, head of, vp, engineering manager, sales, recruiter...)	Checked first, same whole-word match. "lead" and "manager" are deliberately left out because they're ambiguous.
Location must not be only outside the US	A job is dropped only if *every* location it lists is on the exclude list, so "London; New York" passes. Blank locations pass.
Priority (scoring order only)	+3 entry-level title word, +1 target role, +1 from Simplify/sweep, −2 higher-level word, −3 if the description asks for 3+ years. Scoring takes the highest priority first, so the nightly 60-job cap goes to the best bets. Priority never removes a job.

3 · The agents

An "agent" here is one Claude call with one job. Every agent runs through the same function, `scorer.call_claude`, which starts `claude -p` (Claude Code in print mode) with your plan's login. The settings are locked down:

- `--tools ""`: no tools at all. An agent can only read the prompt and answer, so it can't touch files or the web.
- `--safe-mode`: ignores CLAUDE.md, skills, hooks and MCP, so nothing from your Claude Code setup leaks into scoring.
- `--output-format json` + `--no-session-persistence`: a JSON reply, and no saved session for each one-off call.
- The prompt goes in on **stdin** because Windows caps a command line at ~32K characters. `CREATE_NO_WINDOW` stops console windows from flashing during the nightly run.

Agent	Model	Reads	Job	Output and what checks it
Scorer `scorer.py`	Sonnet (nightly; pick in app)	`profile.md`, `feedback.md`, your rubric, 10 job descriptions (each capped at 15,000 chars)	Score each job 1-10 for fit with a reason, matched skills, gaps, and any **hard gates** quoted word for word from the posting.	JSON per job. Code validates types, checks every gate quote against the posting, and applies caps.
Condense `condense.py`	default	Current `feedback.md` + jobs where your rating differs from Claude's by 2+	Propose an updated, short list of scoring rules that explain the disagreements.	Writes `feedback.proposed.md` only. **You** copy what you agree with.
Tailor `drafter.py`	Opus	`full-profile.md`, the job, your "Why this company?" notes	Write a letter plan, then a one-page resume and a cover letter (see section 5).	JSON draft. `validate_draft` checks shape.
Truth check	Opus	Profile, job, the draft	Audit every claim against the profile and every letter rule. Fix must-fix problems itself; list suggestions.	Corrected draft + issue list, shown under "Review notes".
Letter reviewer	Opus	Profile, job, the letter as you've edited it	Critique only (changes nothing): 0 specific ties to the job, 1 story fit, 2 accuracy, 2b forced connections, 3 voice, 4 flow.	Issues with severity, quote and suggestion, shown in the app.

Reviser	Opus	Profile, job, letter, the feedback you keep or write	Rewrite the cover letter only, following your feedback.	New letter; resume and header setting kept as they were.

How a score is decided: hard gates

Some requirements disqualify you no matter how good the rest of the fit is. The scorer reports these as gates, each with an exact quote from the posting. Python then searches for that quote in the description (after normalizing case, curly quotes and spaces). **Only a verified quote caps the score.** An invented quote is shown but ignored, because a false gate would bury a real match.

Gate	Max score
Required 5+ years	3
Required 3-4 years	5
Active security clearance	4
License / cert / advanced degree you lack	4
People-management experience	4
Non-English fluency	4
Must be outside the US	4

Two numbers are saved. `fit_score` is what Claude thought. `score` is after the caps. So you can see when a great fit was capped by a hard requirement.

Prompt versioning. Every score records `PROMPT_VERSION` (now **v4**) and the model used, so scores from an older prompt can be told apart and re-scored.

Defensive parsing. A score must be an integer 1-10 (not 7.5, not `true`). A malformed reply fails only that job, which is logged and retried on a later run.

4 · Your feedback loop for scoring



Not used yet. There are 0 ratings so far and no `feedback.md` file, so the scorer runs on `profile.md` and the rubric alone. Rating 15-20 jobs (especially ones where you disagree with the score) is what turns this loop on.

5 · Drafting a resume and cover letter

From any job page you click **Draft** and can add your own "Why this company?" notes. A background task then runs:

1. **Tailor (Opus)** reads `full-profile.md`, your master profile with every fact, story and voice sample, plus the posting. It returns a JSON draft that includes a **letter plan**: for every paragraph, the exact words from the posting it answers and why that makes you a great candidate.
2. **Truth check (Opus)** compares every claim with the profile and every letter rule, fixes must-fix problems itself, and lists suggestions.
3. **Code checks** (no AI) remove any skill that isn't in the profile, flag banned phrases, and count em dashes.

4. The draft is saved, and the application page shows the plan table, review notes, and an editor for every field.
5. You edit. Optionally click **Get feedback on the letter** (reviewer agent), trim the suggestions to the ones you agree with, and click **Revise the letter** (reviser agent).
6. **Save & generate PDFs**: the draft fills HTML templates (Cambria), headless Edge prints them to PDF, and Python reads the page count from the PDF. If the resume runs past one page, it tries again at 10.5pt, then 10pt. Files go to `applications/<Company - Role>/`.

Where your writing voice comes from

`full-profile.md` has two kinds of content, and the prompts treat them differently:

Voice samples (imitate these)
Only letters you wrote entirely yourself: **Oscar** (your current style, and the structure to follow), **Google** (your sentence habits), and **Cosm**. Stories saved "in my words" are your exact text.

Notes and facts (use, never imitate)
Everything else: projects, jobs, stories with "Good for" fit notes, lessons from feedback rounds, the AI-isms list. The prompt says plainly that its wording is not your voice.

Provenance rule: AI-written or AI-edited text is never saved as your voice or labeled "my words." This keeps letters from drifting toward AI style over time.

The cover letter shape

¶	What it does
1	Open with a story from your life that fits *this* company (lifeguarding only for healthcare or safety), stated plainly, tied to a named value or requirement.
2	What the story shows about you, tied to what the role asks.
3	One relevant project told through motivation: why you built it, the struggle, what changed for you. Name the tech overlap with the posting ("built it in Python") and end on the goal it helped you reach, tied to the job or the company's values.
4	Close on the company's culture and mission, with your personal reason and real excitement to apply.

About 450-500 words. Paragraphs don't have to connect to each other, but every paragraph must connect to the job.

6 · Every rule, and how it's enforced

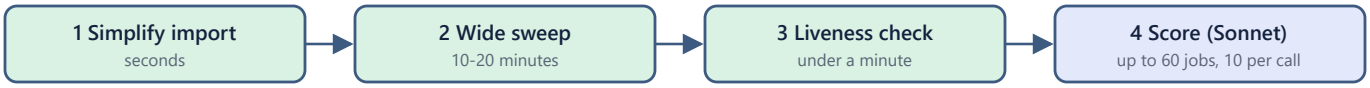
Rules are enforced in different ways, and the difference matters. Code rules are guaranteed every time. Prompt rules are instructions to Claude: it follows them most of the time, but nothing forces it, which is why the important ones are also checked by a second agent or by you.

Rule	Enforced by	Where and how
Truthfulness		
Only facts from your profile; no invented numbers, tools, results or team sizes	Prompt ×2	Tailor's first section, then the truth check audits every claim and fixes violations.
Don't link facts with cause and effect the profile doesn't state	Prompt ×2	Tailor rule + truth check.

Skip anything marked [confirm]	Prompt	Tailor rule. Unverified facts stay out until you confirm them.
Every resume skill must exist in your profile	Code	`code_checks` removes any skill whose parts aren't found in `full-profile.md` and tells you which.
Never mention missing skills or "eager to learn"	Prompt + Code	Tailor rule; regexes in `BANNED_PHRASES` flag "lack of experience", "quick learner", etc.
The cover letter		
Show why you're a GREAT candidate: every paragraph tied specifically to a named line in the posting	Prompt ×3 + You	Tailor must write the letter plan first; truth check treats vague ties as must-fix; reviewer checks it first, as high severity. The plan table in the app lets you check each tie.
Technical paragraphs name the tech overlap and end on a goal tied to the job	Prompt ×3	Tailor, truth check (must-fix), reviewer.
No em dashes, ever	Prompt + Code	In every prompt; `em_dash_count` warns after drafting and when you save edits.
No AI-isms ("leverage", "seamless", "not just X but Y", "testament to", "journey"...)	Prompt + Code	AI-isms list in the profile + `BANNED_PHRASES` regex warnings.
Stories fit the company (lifeguard only for health/safety)	Prompt ×3	"Good for" notes on each story; truth check and reviewer flag misfits.
Only relevant projects; security coursework only for security roles; no bug stories	Prompt ×3	Tailor story rules; truth check must-fix; reviewer "focus" check.
No invented bridges between paragraphs	Prompt ×3	Tailor, truth check, reviewer check 2b.
Voice from your own letters only; never polish your phrasing	Prompt + You	Prompt names the 3 sample letters; truth check has a "never fix his voice" rule. Saving AI text as voice is banned (memory rule).
~450-500 words	Prompt	Tailor rule; the editor shows a live word count.
The resume		
Exactly one page	Prompt + Code	Word limits per line in the prompt; render loop reads `/Count` from the PDF and steps the font 11 → 10.5 → 10pt.
Scoring and collection		
Hard requirements cap the score	Code	Gate quotes verified in the posting text, then `GATE_CAPS` applied.
filters.yaml edits always apply before scoring	Code	`refilter_all()` runs at the start of every scoring run.
Agents can't write feedback.md	Code + You	Condense only writes `feedback.proposed.md`.
Nightly can't drain your Claude plan	Code	60-job cap per night in `nightly.yaml`; the rest waits.
Be polite to job boards	Code	Delays between requests; sweep limited to 8 at a time; dead boards cached and skipped.
No API keys in code	Code	All Claude calls go through the logged-in `claude` CLI; no key is stored anywhere.

7 · The nightly run

A Windows Task Scheduler task, **"Job Sourcer Nightly"**, starts `nightly.py` at 2 AM with `pythonw` (no window). It can wake the PC, and if the PC was off it runs as soon as it's back on. A lock file stops two runs from overlapping. Each step is switched on or off in `nightly.yaml`, and one failing step doesn't stop the others. The Fetch page shows the last run.



8 · The web app

`start.bat` launches a Flask server on your PC and opens it in the browser. Long jobs (scoring, import, sweep, drafting) run in a `BackgroundTask` thread while the page polls for progress, so the browser never times out. **After any change to a `.py` file, close and re-run `start.bat`**: the running server keeps the old code.

Page	What it's for
Jobs	Views: scored, to score, filtered out, no description, closed, applied. Sort and filter by score or company.
Job detail	Score, reason, matched skills, gaps, gates, the description, the tracker, and the Draft button.
Rate	A queue to rate jobs 1-10 with a reason.
Fetch	Fetch one board, run the Simplify import or sweep, check liveness, see the last nightly run.
Score	Preview the exact prompt, pick the model and limit, start or stop scoring.
Feedback	Run condense and view proposed scoring rules.
Applications	Every draft and tracked job, applied ones first; each opens the editor, letter plan, feedback and PDFs.

9 · Where things live

File	Role
`profile.md`	Short profile the **scorer** reads
`full-profile.md`	Master profile the **drafter** reads: facts, stories, voice
`feedback.md`	Your approved scoring rules (not created yet)
`filters.yaml`	Title, location and priority rules
`feeds.yaml`	Simplify categories, sweep keywords
`nightly.yaml`	Nightly steps, cap, model
`data/jobs.db`	Everything else (SQLite)
`applications/`	Generated PDFs
`boards/`	~15,900 board slugs for the sweep

Table	Holds
`jobs`	Every posting: status, score, fit_score, reason, gates, priority, closed_at, app_status, applied_at
`ratings`	Your 1-10 ratings and reasons
`applications`	Draft JSON, review notes, model, rendered PDF paths
`application_events`	Tracker history (applied, interview, offer...)

New columns are added with `ALTER TABLE` on startup (the `ADDED_COLUMNS` list), so old databases upgrade themselves without losing data.

10 · Open items

- **[confirm] facts in full-profile.md** are kept out of drafts until you answer them: GPA, Durbin-Watson ("re-validated"), whether the NYC project was for a class, your part in SQ3R, the music project names, the Contagion Model's course and language, the scout teaching story in your own words, First Aid/CPR.
- **Rating loop unused:** 0 ratings and no feedback.md yet (section 4).
- **profile.md anchor examples** for scoring were drafted by Claude and marked DRAFT. They're worth a read-through.
- **Untested:** "Revise the letter" hasn't been run on a real letter, and no draft has been written yet under the new "GREAT candidate" rules.
- **Small mismatch:** the editor's word count hint says "aim for 300-400," but the letter rule is 450-500.